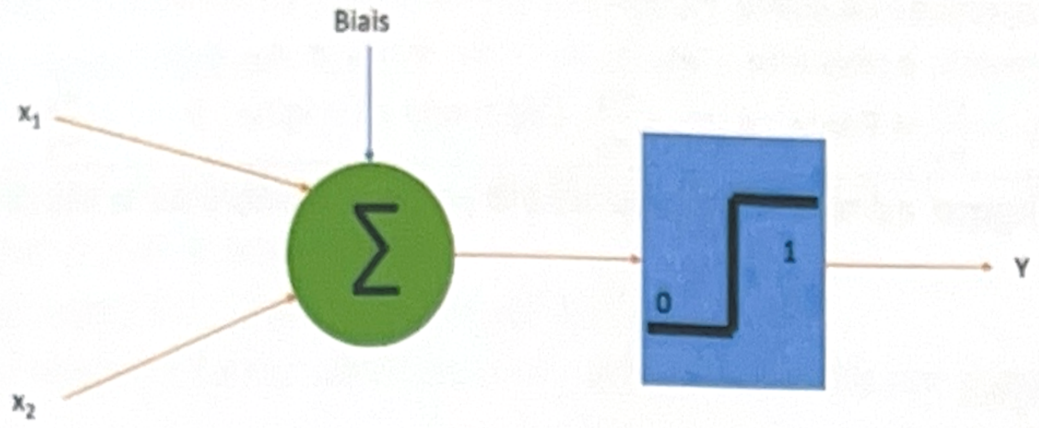
# Les réseaux de neurones

## 1. Développement d'un perceptron simple :

### i. Contexte

Un **perceptron simple** est le modèle de base des réseaux de neurones, utilisé pour la classification binaire. Il fonctionne comme un classificateur linéaire en prenant un ensemble de données en entrée, appliquant une fonction de pondération, puis utilisant une fonction d'activation pour produire un résultat (souvent 0 ou 1).




### ii. Structure d'un perceptron simple

- **Entrées (Input)** : Chaque neurone reçoit plusieurs entrées, représentées par un vecteur **($x_1$, $x_2$, ..., $x_n$)**. Ce sont les caractéristiques de l'exemple à classifier.
- **Poids (Weights)** : Chaque entrée est associée à un poids **$w_i$** qui reflète l'importance de cette entrée pour la décision finale. Ces poids sont ajustés pendant l'entraînement.
- **Biais (Bias)** : Un terme biais $\theta$ est ajouté pour décaler la fonction de décision afin de rendre le modèle plus flexible.
- **Somme pondérée** : Le perceptron calcule une **somme pondérée des entrées et des poids**, plus le biais :

$$\mathbf{out} = x_1 * w_1 + x_2 * w_2 + \cdots + x_n * w_n + \theta$$

- **Fonction d'activation** : Pour déterminer la sortie, le perceptron utilise **une fonction d'activation**, souvent une **fonction seuil** (ou fonction sigmoïde dans certains cas), qui prend la somme pondérée et la transforme en une sortie binaire.

$$out = \begin{cases} 0 \; si \; z < 0 \\ 1 \; si \; z \geq 0 \end{cases}$$

### iii. Limitations :

Le perceptron simple ne peut traiter que des problèmes **linéairement séparables.** Autrement dit, pour qu'il fonctionne correctement, il doit exister une ligne droite — ou, dans des espaces de dimension supérieure, un hyperplan — capable de séparer les points appartenant à des classes différentes.

### *1.1. Mise en place d'un patron simple*

- *Créer une fonction : **[y]=perceptron simple(x,w,active).***

```
import numpy as np # importe la bibliothèque NumPy, utilisée pour les calculs mathématiques et les vecteurs.
import matplotlib.pyplot as plt # sert à tracer des graphiques (exemple : séparations de classes, courbes).

def perceptron_simple(x, w, active):
    x_aug = ..... # Ajouter le biais à l'entrée (1 au début du vecteur x). Cela donne un vecteur augmenté x_aug = [1, x1, x2] compatible avec les 3 poids du vecteur w. ⟶ But : permettre de traiter le biais comme un poids normal (w[0] * 1) dans le produit scalaire.

    # Calculer la somme pondérée : On effectue le produit scalaire entre le vecteur des poids w et l'entrée augmentée x_aug.
    z = ..... # Cela calcule la somme pondérée : z=w0·1+w1·x1+w2·x2

    # Fonction d'activation
    if ..... == 0:
        y = ..... # Fonction sign : Elle retourne : -1 si z < 0 ; 0 si z == 0 (rare en pratique) ; +1 si z > 0
    elif active == 1:
        y = ..... # Fonction tanh : renvoie une valeur continue entre -1 et 1 selon z
    else:
        raise ValueError("active doit être 0 (sign) ou 1 (tanh).") # erreur ⟶ pour éviter un comportement inattendu.
    return y # Retourne la sortie finale du perceptron, après activation. C'est le résultat de la prédiction pour ce vecteur d'entrée x.
```

- *Tester votre perceptron avec l'exemple du **OU logique** (en utilisant **φ(x) =sign(x)**).*
- *Afficher dans le cadre de l'exemple du **OU logique** sur la même figure les différents éléments de l'ensemble d'apprentissage*

*droite séparatrice associée aux poids du neurone sur la même figure.*

- ✓ **Perceptron simple :** Entrée **x = [x₁, x₂]**, poids **w = [w₀, w₁, w₂]** (où **w₀ : biais**, **w₁** et **w₂** : poids associés à **x₁** et **x₂**).
- ✓ **Somme pondérée** (avant l'activation) est donnée par : **z = w₀ + w₁ * x₁ + w₂ * x₂**
- ✓ Le perceptron va prédire une classe en fonction du signe de **z** après avoir appliqué la fonction d'activation (**'sign'** : cas OR logi
- ✓ **Équation de la droite séparatrice :** La droite séparatrice correspond à la frontière entre les deux classes du perceptron. E

trouve lorsque **z = 0**. Cela revient à résoudre : $w_0 + w_1 * x_1 + w_2 * x_2 = 0$

En isolant $x_2$ : $x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2} \Rightarrow$ Pente : $a = -\frac{w_1}{w_2}$ ; Ordonnée à l'origine : $b = -\frac{w_0}{w_2}$

**# Fonction 'test_perceptron_ou_logique' qui teste le comportement du perceptron simple sur le cas du OU logique.**

```
def test_perceptron_ou_logique():

    # Définition des données du OU logique
    X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Entrées
    Y = .....                                       # Sorties attendues

    # Définition des poids du perceptron (biais + w1 + w2) : choisis ici manuellement pour que le perceptron apprenne la fonction OU logique.
    w = .....  # exemple : w[0] = -0.5 → biais (θ) du neurone.  w[1] = 1 et w[2] = 1 → poids associés à x1 et x2.
    print("Test du perceptron sur l'exemple du OU logique :") # Affiche un titre pour le test dans la console.

    for .....  # Boucle sur les 4 entrées possibles de X
        y_pred = .....  # On appelle 'perceptron_simple' pour prédire la sortie à l'aide des poids w et de la fonction sign.

        # Affiche pour chaque entrée : L'entrée utilisée, La sortie prédite par le perceptron, La sortie correcte (attendue) selon la table de vérité.
        print(f"Entrée: {X[i]} - Sortie prédite : {y_pred} - Sortie attendue: {Y[i]}")

    afficher_resultats(X, Y, w) # Fonction pour afficher les points et la droite séparatrice

# Fonction pour afficher les points et la droite séparatrice
def afficher_resultats(X, Y, w):

    # On trace les points de chaque classe
    plt.scatter(.....  ) # En rouge ceux de classe 0 (sortie attendue = 0)
    plt.scatter(.....  ) # En vert ceux de classe 1 (sortie attendue = 1)

    # Tracer la droite séparatrice : z=a*t+b
    t = [-0.5, 1.5]       # On choisit deux valeurs de x1 (abscisse) pour tracer la droite.
    a = .....     # Pente de la droite (calculée à partir des poids)
    b = .....     # Ordonnée à l'origine (intersection avec l'axe y)
    z = .....  # deux points pour tracer la droite

    # Tracé de la droite de séparation (en bleu), représentant la frontière entre les classes selon le perceptron.
    .....

    .....

    .....

# Q 1.1
test_perceptron_ou_logique()
```

## 1.2. Etude de l'apprentissage

- Créer une fonction : **w, erreur=apprentissage_widrow(x, yd, epoch, batch size)**
- La droite réparatrice et les points d'apprentissage seront affichés à chaque itération (une itération correspond à la présentation de tous les individus de l'ensemble d'apprentissage), ainsi que l'erreur de classification. L'algorithme s'arrêtera dès que **l'erreur est nulle**, ou **au bout de Epcoh itérations**. Les poids initiaux seront générés de manière aléatoire.

```
def apprentissage_widrow(x, yd, epoch, batch_size): # Fonction principale d'apprentissage, avec comme entrée :
# x : données d'entrée (2 lignes × n colonnes).    # yd : étiquettes cibles (+1 ou -1).  # epoch : nombre d'itérations sur tout l'ensemble.
# batch_size : nombre d'échantillons avant chaque mise à jour des poids.

    n_samples = .....          # Récupère 'n_samples' : le nombre d'exemples d'apprentissage (colonnes de x).
    w = ..... # Initialise aléatoirement le vecteur de poids 'w' avec 3 valeurs :
            # w[0] = biais (b).     w [1] = poids associé à x1.      w[2] = poids associé à x2.)
    erreur = []  # Initialise une liste vide pour enregistrer l'erreur cumulative 'erreur' à chaque epoch (utile pour observer la convergence).

    # Ajoute une ligne de 1 à l'ensemble d'apprentissage x pour gérer le biais dans le produit matriciel
    x_biais = .... # 'x_biais' devient une matrice de forme (3×n_samples), où la première ligne est 1 pour le biais.

    # Apprentissage sur plusieurs epochs
    for ep in range(.....): # Boucle principale sur les epoch (itérations sur tout l'ensemble).
        erreur_epoch = 0  # Initialise l'erreur cumulée 'erreur_epoch' à 0 pour l'epoch en cours.

    # Traiter les données par petits groupes (batches) :
        for i in range(.....): # C'est une fonction range() qui génère une séquence de nombres entiers allant de 0 (inclus) → point de départ, # jusqu'à
n_samples (exclus) → la fin, # avec un pas de batch_size

            # Sélectionne le sous-ensemble (batch) de données et cibles correspondant.
            x_batch = .....
            yd_batch = .....

            # Calcule la prédiction du perceptron : Produit scalaire w^T·x pour chaque échantillon.
            y_pred = ..... # Utilise sign pour prédire +1 ou -1 (fonction d'activation seuil).
            # y_pred[y_pred == 0] = 1  # correction : éviter 0 comme sortie

            # Calcul de l'erreur 'erreur_batch' pour chaque élément du batch
            erreur_batch = ..... # erreur = réponse attendue - prédiction.

            # Mise à jour des poids par descente de gradient
            # Formule : w = w + α * Δw où : α est pris implicitement comme 1 / batch_size, Δw est obtenu en multipliant les erreurs par les entrées.
            w += ..... # np.dot(erreur_batch, x_batch.T) = somme des gradients sur le batch.

            # On ajoute l'erreur quadratique du batch à l'erreur totale de l'epoch ⟶ erreur = ∑_{i=0}^{N-1} (y_d^(i) − y^(i))^2
            erreur_epoch += .....

    # Fonction pour tracer la droite séparatrice et les points d'apprentissage d'une epoch.
    affiche_classe(w, x, yd, f"Epoch {ep} (erreur: {erreur_epoch})")

    # Enregistre l'erreur cumulative de l'epoch dans la liste erreur.
    erreur.append(erreur_epoch)
```

```
# Arrêter l'apprentissage si l'erreur est nulle
if erreur_epoch == 0:
    break

return w, erreur # Retourne les poids finaux appris et le vecteur des erreurs pour pouvoir tracer l'évolution de l'apprentissage.

# Fonction pour tracer la droite séparatrice et les points d'apprentissage d'une epoch.
def affiche_classe(w, x, yd, titre):
    # Tracer les points de l'ensemble d'apprentissage
    for i in range(.....):
        if yd[i] == 1:
            plt.scatter(.....) # Classe +1 : vert, cercle.
        else:
            plt.scatter(.....) # Classe -1 : rouge, croix

    if w is not None and w[2] != 0:
        # Tracer la droite séparatrice : z=a*t+b
        t = [.....] # Intervalle pour x1
        a = ..... # Pente de la droite
        b = ..... # Ordonnée à l'origine
        z = [.....] # Calcule les deux points (x1, x2) de la droite.
        plt.plot(.....) #Trace la droite en bleu.
    else:
        print("Impossible de tracer la droite (division par zéro).")

    plt.title(titre)  # ajout de titre
    plt.axis([.....])  # définir les limites des axes du graphique à tracer
    plt.grid(True) # ajoute une grille pour faciliter la lecture
    plt.show() # show() : affiche le graphique à l'écran
```

### 1.2.2 Test 1 simple

*— Charger des données p2 d1.txt.*

*— Appliquer votre algorithme d'apprentissage. Afficher l'évolution de l'erreur. Vérifier que la frontière est correcte*

.....

.....

.....

.....

### 1.2.3 Test 2

*— Charger des données p2 d2.txt.*

*— Appliquer votre algorithme d'apprentissage de Widrow-Hoff. Comparer avec les premières données et conclure.*

.....

.....

.....

.....

## 1.3. Perceptron multicouches

### 1.3.1. Mise en place d'un perceptron multicouche

```
def multiperceptron(x, wl, w2):
    # Ajouter le biais à l'entrée
    x_biais = np.array([.....])  # 1 pour le biais, x[0] et x[1] sont les entrées

# Calcul des sorties de la couche cachée
z_h1 = ..... # Sortie du 1er neurone de la couche cachée
z_h2 = ..... # Sortie du 2ème neurone de la couche cachée

 # Application de la fonction d'activation (sigmoïde)
 a_h1 = .....
 a_h2 = .....

 # Créer un vecteur d'entrée pour la couche de sortie
 a_h = ..... # Ajouter le biais pour la couche de sortie

 # Calcul de la sortie du neurone de la couche de sortie
 z_out = .....

 # Application de la fonction d'activation pour la sortie
 y = .....   return y
# Q 1.3.1
 wl = np.array([..... ])
 w2 = np.array([.....]).T
 y = multiperceptron(.....)
 print(f"y: {y}")
```

### 1.3.2. Programmation apprentissage multicouches

```
def multiperceptron_widrow(x, yd, Epoch, batch_size):
# Initialiser les poids synaptiques de manière statique
wl = np.array([..... ])
w2 = np.array([.....])  # Poids pour la couche de sortie (biais inclus)
alpha = 0.5  # Taux d'apprentissage
erreur = []  # Liste pour stocker l'erreur cumulée

 # Boucle d'apprentissage pour Epoch itérations
 for epoch in range(.....):
     # Parcours par lots (batch)
     for i in range(.....):
         x_batch = x[.....]
         yd_batch = yd[.....]

         # Initialiser les erreurs pour le batch
total_error = 0

for j in range(.....): # nombre lignes – échantillons dans un batch
    # Entrée avec biais
    x_biais = np.array([.....]) # Biais + entrées
```

```
# Calcul des sorties de la couche cachée
z_h1 = ....
z_h2 = ....

# Fonction d'activation (sigmoïde)
a_h1 = ....
a_h2 =....

# Entrée pour la couche de sortie
a_h = ....

# Calcul de la sortie de la couche de sortie
z_out = ....

# Application de la fonction d'activation pour la sortie
y_pred = ....

# Calcul de l'erreur
error = ....
total_error += ....

# Mise à jour des poids de la couche de sortie
w2 += ....

# Mise à jour des poids de la couche cachée
delta_h = ....
w1[:, 0] += ....
w1[:, 1] += ....

# Ajout de l'erreur moyenne pour le batch à la liste
erreur.append(total_error / batch_size)

# Affichage de l'erreur cumulée à chaque itération
print(f"Epoch {epoch + 1}/{Epoch}, Erreur cumulée: {total_error / batch_size:.4f}")

# Condition d'arrêt si l'erreur est nulle
if total_error == 0:
    break
return w1, w2, erreur
```

*— Créer l'ensemble d'apprentissage. Afficher cet ensemble avec la fonction affiche classe. Pensez-vous que ce problème puisse être traité par un perceptron simple ?*

*— Appliquer votre algorithme d'apprentissage.*

*—Tester, à partir de votre fonction* ***multiperceptron****, le réseau de neurones ainsi obtenu sur l'ensemble d'apprentissage. Appliquer votre algorithme d'apprentissage.*

*—Afficher les droites s´séparatrices associées aux différents neurones et les points de l'ensemble d'apprentissage.*